AMRITA VISHWA VIDYAPEETHAM | School of Engineering Chennai Campus

Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

KANTAMRAJU SREEJA

CH.SC.U4CSE24255

WORKSHEET-4

WORKSHEET- 4

EX.NO : 4

DATE : 10-10-26

TITLE : Quantum Data Encoding, Quantum Kernels and Quantum Clustering / QSVM Using Qiskit

AIM:

To encode classical data into quantum states using basis, angle and amplitude encoding in Qiskit, to compute a quantum kernel from the overlap of encoded states, and to use it for classification (QSVM) and clustering on a small dataset, with a comparison against a classical method.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit 2.x and Qiskit Aer simulator (`pip install qiskit qiskit-aer`)
- Matplotlib and pylatexenc (for circuit diagrams and histograms)
- NumPy and scikit-learn (`pip install numpy scikit-learn`)
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

A quantum circuit cannot read classical numbers directly. **Data encoding**, also called a feature map, is the step that loads classical data into a quantum state. Three basic methods are used.

Basis encoding stores each bit in one qubit: an X gate is applied wherever the bit is 1. The bit string 1011 becomes the state $|1011\rangle$ and needs four qubits.

Angle encoding stores each feature as a rotation angle, using one qubit per feature. A value is first scaled to the range 0 to 1 with min-max normalisation, $x = (x_{raw} - x_{min})/(x_{max} - x_{min})$. It is then converted to the angle $\theta = \pi x$ and applied with an R_y gate:

$$R_y(\theta)|0\rangle = \cos\frac{\theta}{2}|0\rangle + \sin\frac{\theta}{2}|1\rangle, \qquad P(1) = \sin^2\frac{\theta}{2}$$

For example, 50 marks out of 100 gives $x = 0.5$, $\theta = /2$ and $P(1) = 50\%$. The amplitudes use half the angle, so $P(1)$ is not equal to the data value in general: $x = 0.25$ gives $P(1) = 14.6\%$.

Amplitude encoding stores a vector of N values in the amplitudes of $\log_2 N$ qubits. The vector is padded with zeros to a power of two and divided by its length (norm). The vector (3,4) becomes $0.6|0\rangle + 0.8|1\rangle$. Few qubits are needed, but the circuit that prepares the state is deep, so loading data in this way is not free.

Quantum kernel. A kernel measures how similar two data points are. Once two points x and x' are encoded as the states $|\phi(x)\rangle$ and $|\phi(x')\rangle$, their similarity is the squared overlap (fidelity) of the two states:

$$k(x, x') = |\langle\phi(x)|\phi(x')\rangle|^2$$

The kernel is 1 for identical points and falls towards 0 as the states become different. The values for all pairs of points form the kernel matrix. On a simulator the overlap can be computed exactly from the two statevectors. On hardware it is estimated with the **swap test**: an extra qubit is put into superposition with a Hadamard gate, controls a swap of the two states, and receives a second Hadamard gate. It then measures 0 with probability

$$P(0) = \frac{1}{2} + \frac{1}{2}|\langle a|b\rangle|^2$$

QSVM. A support vector machine needs only the kernel matrix of the data. In a quantum support vector machine the quantum circuit supplies the kernel matrix, and a classical SVM (for example `SVC(kernel="precomputed")` in scikit-learn) does the training and the prediction. **Quantum clustering** uses the same idea without labels: points are grouped by the fidelity between their encoded states, either by assigning each point to the nearest centre with the distance $1 - k$, or by passing the kernel matrix to a clustering method.

A quantum kernel is not guaranteed to be more accurate than a classical one. On the small datasets used here, the purpose is to understand the workflow and to compare the result honestly with a classical method.

Bit ordering in Qiskit. Qiskit prints qubit 0 as the rightmost bit, so place the X gates of a basis encoding while looping over the reversed bit string. The printed state then matches the string that was written.

PROCEDURE:

1. Install the required packages with `pip install qiskit qiskit-aer matplotlib pylatexenc numpy scikit-learn`, and print the installed version with `qiskit._version_`.

2. Open Jupyter Notebook or Visual Studio Code.

3. Import `QuantumCircuit` and `transpile` from `qiskit`, `Statevector` and `Operator` from `qiskit.quantum_info`, `AerSimulator` from `qiskit_aer` and `plot_histogram` from `qiskit.visualization`. Import NumPy and the required scikit-learn modules.

4. Scale every feature to the range 0 to 1 with min-max normalisation before encoding it.

5. Encode the data: X gates for basis encoding, `qc.ry(np.pi * x, i)` for angle encoding (angles are in radians) and `qc.initialize(...)` for amplitude encoding.

6. Draw the circuit with `qc.draw("mpl")` before running it. Display the statevector and the probabilities with `Statevector(qc)`.

7. Write a function that returns the encoded state of a data point, and a second function that returns the kernel value for two data points as the squared overlap of their states.

8. Build the kernel matrix for the training data and for the test data against the training data.

9. Train a classical SVM with the precomputed kernel, or cluster the data using the kernel matrix, and compare the result with a classical method on the same data.

10. State the encoding and the scaling rule used with every result. Record the code, the circuit diagram, the output and a one-line inference under each program.

PROGRAM:

Program 1: Encode the bit string 1011 using basis encoding. Verify that the printed state is 1011. Repeat with a four-bit string of your own choice.

OUTPUT:

```
def basis_encode(bitstring):
    qc = QuantumCircuit(len(bitstring))
    for i, bit in enumerate(bitstring):
        if bit == '1':
            qc.x(i)
        elif bit != '0':
            raise ValueError("Bit string must contain only 0 and 1")
    return qc

for bits in ['1011', '0110']:
    qc = basis_encode(bits)
    state = Statevector.from_instruction(qc)
    index = int(bits, 2)
    print("Input bit string:", bits)
    print("Prepared computational-basis state:", format(index, f'0{len(bits)}b'))
    print("Statevector:", state)
    print(qc.draw('text'))
    print()
```

```
Input bit string: 1011
Prepared computational-basis state: 1011
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
             0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j,
             0.+0.j, 0.+0.j],
            dims=(2, 2, 2, 2))

q_0: ┤ X ├
q_1: ─────
q_2: ┤ X ├
q_3: ┤ X ├


Input bit string: 0110
Prepared computational-basis state: 0110
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j,
             0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
             0.+0.j, 0.+0.j],
            dims=(2, 2, 2, 2))

q_0: ─────
q_1: ┤ X ├
q_2: ┤ X ├
q_3: ─────
```

Program 2: Encode one feature using angle encoding: 50 marks on a scale of 0 to 100. Display the statevector and the probabilities. Repeat for x = 0, 0.25, 0.5, 0.75 and 1 and tabulate θ and $P(1)$.

OUTPUT:

```python
def angle_encode_one(x):
    if not 0 <= x <= 1:
        raise ValueError("x must be between 0 and 1")
    theta = np.pi * x
    qc = QuantumCircuit(1)
    qc.ry(theta, 0)
    return qc, theta

qc, theta = angle_encode_one(0.5)
state = Statevector.from_instruction(qc)
probs = state.probabilities()
print("50 marks: x = 0.5, theta =", theta)
print("Statevector:", state)
print("Probabilities [P(0), P(1)]:", probs)
print(qc.draw('text'))

print("\nx | theta (radians) | P(1)")
for x in [0, 0.25, 0.5, 0.75, 1]:
    _, theta = angle_encode_one(x)
    p1 = np.sin(theta / 2) ** 2
    print(f"{x:.2f} | {theta:.4f} | {p1:.4f}")
```

```
50 marks: x = 0.5, theta = 1.5707963267948966
Statevector: Statevector([0.7071+0.j, 0.7071+0.j],
             dims=(2,))
Probabilities [P(0), P(1)]: [0.5 0.5]
q: ┤ Ry(π/2) ├

x | theta (radians) | P(1)
0.00 | 0.0000 | 0.0000
0.25 | 0.7854 | 0.1464
0.50 | 1.5708 | 0.5000
0.75 | 2.3562 | 0.8536
1.00 | 3.1416 | 1.0000
```

Program 3: Encode a four-feature data sample (for example one row of the Iris dataset, scaled to 0 to 1) using angle encoding with one qubit per feature. Draw the circuit and display the probabilities.

OUTPUT:

```
iris = load_iris()
raw_sample = iris.data[0]
sample = MinMaxScaler().fit_transform(iris.data) [0]
qc = QuantumCircuit(4)
for i, value in enumerate(sample):
    qc.ry(np.pi * value, i)

state = Statevector.from_instruction(qc)
print("Original Iris sample:", raw_sample)
print("Scaled features:", sample)
print(qc.draw('text'))
print("Statevector:", state)
print("Probabilities:", state.probabilities())
```

```
Original Iris sample: [5.1 3.5 1.4 0.2]
Scaled features: [0.2222 0.625  0.0678 0.0417]
     ┌──────────┐
q_0: ┤ Ry(2π/9) ├
     ├──────────┤
q_1: ┤ Ry(5π/8) ├
     ┌──────────┐
q_2: ┤ Ry(0.21299) ├
     └──────────┘
     ├──────────┤
q_3: ┤ Ry(π/24) ├
     └──────────┘
Statevector: Statevector([0.518 +0.j, 0.1885+0.j, 0.7752+0.j, 0.2822+0.j, 0.0554+0.j,
             0.0202+0.j, 0.0829+0.j, 0.0302+0.j, 0.034 +0.j, 0.0124+0.j,
             0.0508+0.j, 0.0185+0.j, 0.0036+0.j, 0.0013+0.j, 0.0054+0.j,
             0.002 +0.j],
            dims=(2, 2, 2, 2))
Probabilities: [0.2683 0.0355 0.601  0.0796 0.0031 0.0004 0.0069 0.0009 0.0012 0.0002
 0.0026 0.0003 0.     0.     0.     0.    ]
```

Program 4: Encode the vectors (3, 4) and (1, 2, 3, 4) using amplitude encoding with `qc.initialize`. Verify that the probabilities are the squares of the normalised values, and use `qc.decompose(reps=6)` to show the gates that the preparation really uses.

OUTPUT:

```python
def amplitude_encode(vector):
    vector = np.asarray(vector, dtype=float)
    norm = np.linalg.norm(vector)
    if norm == 0:
        raise ValueError("Vector must not be zero")
    normalized = vector / norm
    nqubits = int(np.ceil(np.log2(len(normalized))))
    dimension = 2 ** nqubits
    padded = np.zeros(dimension)
    padded[:len(normalized)] = normalized
    qc = QuantumCircuit(nqubits)
    qc.initialize(padded, range(nqubits))
    return qc, padded

for vector in [(3, 4), (1, 2, 3, 4)]:
    qc, normalized_padded = amplitude_encode(vector)
    state = Statevector.from_instruction(qc)
    print("Input:", vector)
    print("Normalized (padded if needed):", normalized_padded)
    print("Probabilities:", state.probabilities())
    print("Expected squared normalized values:", normalized_padded ** 2)
    print("Decomposed preparation circuit:")
    print(qc.decompose(reps=6).draw('text'))
    print()
```

```
Input: (3, 4)
Normalized (padded if needed): [0.6 0.8]
Probabilities: [0.36 0.64]
Expected squared normalized values: [0.36 0.64]
Decomposed preparation circuit:
     ┌──────────────┐
q: ─┤0├┤ U(1.8546,0,0) ├
     └──────────────┘

Input: (1, 2, 3, 4)
Normalized (padded if needed): [0.1826 0.3651 0.5477 0.7303]
Probabilities: [0.0333 0.1333 0.3    0.5333]
Expected squared normalized values: [0.0333 0.1333 0.3    0.5333]
Decomposed preparation circuit:
global phase: 5.176
     ┌──────────────────┐┌───┐┌──────────────────┐
q_0: ─┤0├┤ U(π/2,-6.1033,0) ├┤ X ├┤ U(π/2,π/2,-1.1071) ├
     ┌┴──┴┐└──────────────────┘└─┬─┘└──────────────────┘
q_1: ─┤0├┤ U(2.3005,0,0) ├──────■──────────────────────
     └──┘└──────────────┘
```

Program 5: For a four-feature vector, compare basis, angle and amplitude encoding in a table: number of qubits, number of gates and circuit depth.

OUTPUT:

```
features = np.array([0.2, 0.4, 0.6, 0.8])

basis_bits = ''.join('1' if v >= 0.5 else '0' for v in features)
basis_qc = basis_encode(basis_bits)

angle_qc = QuantumCircuit(4)
for i, v in enumerate(features):
    angle_qc.ry(np.pi * v, i)

amplitude_qc, _ = amplitude_encode(features)

def circuit_stats(qc):
    decomposed = qc.decompose(reps=6)
    return qc.num_qubits, decomposed.size(), decomposed.depth()

print("Encoding | Qubits | Gates after decomposition | Depth after decomposition")
for name, circuit in [('Basis', basis_qc), ('Angle', angle_qc), ('Amplitude', amplitude_qc)]:
    print(name, "|", *circuit_stats(circuit), sep=" | ")

print("\nNote: initialize is a high-level state-preparation instruction; its decomposed gate count depends on the Qiskit version and synthesis method.")
```

```
Encoding | Qubits | Gates after decomposition | Depth after decomposition
Basis | | | 4 | 2 | 1
Angle | | | 4 | 4 | 1
Amplitude | | | 2 | 6 | 4

Note: initialize is a high-level state-preparation instruction; its decomposed gate count depends on the Qiskit version and synthesis method.
```

Program 6: For two data points with two features each, compute the quantum kernel value $k(x, x')$ as the squared overlap of their angle-encoded states. Verify that $k(x, x) = 1$.

OUTPUT:

```
def angle_state(x):
    x = np.asarray(x, dtype=float)
    qc = QuantumCircuit(len(x))
    for i, value in enumerate(x):
        qc.ry(np.pi * value, i)
    return Statevector.from_instruction(qc)

def quantum_kernel(x, xp):
    a = angle_state(x).data
    b = angle_state(xp).data
    return float(abs(np.vdot(a, b)) ** 2)

x = [0.2, 0.7]
xp = [0.6, 0.1]
print("x =", x)
print("x' =", xp)
print("k(x, x') =", quantum_kernel(x, xp))
print("k(x, x) =", quantum_kernel(x, x))
```

```
x = [0.2, 0.7]
x' = [0.6, 0.1]
k(x, x') = 0.22612712429686846
k(x, x) = 0.9999999999999996
```

Program 7: Compute the quantum kernel matrix for a small dataset (for example 20 samples from the first two classes of the Iris dataset, using two features). Display the matrix as a heat map and comment on its diagonal and its symmetry.

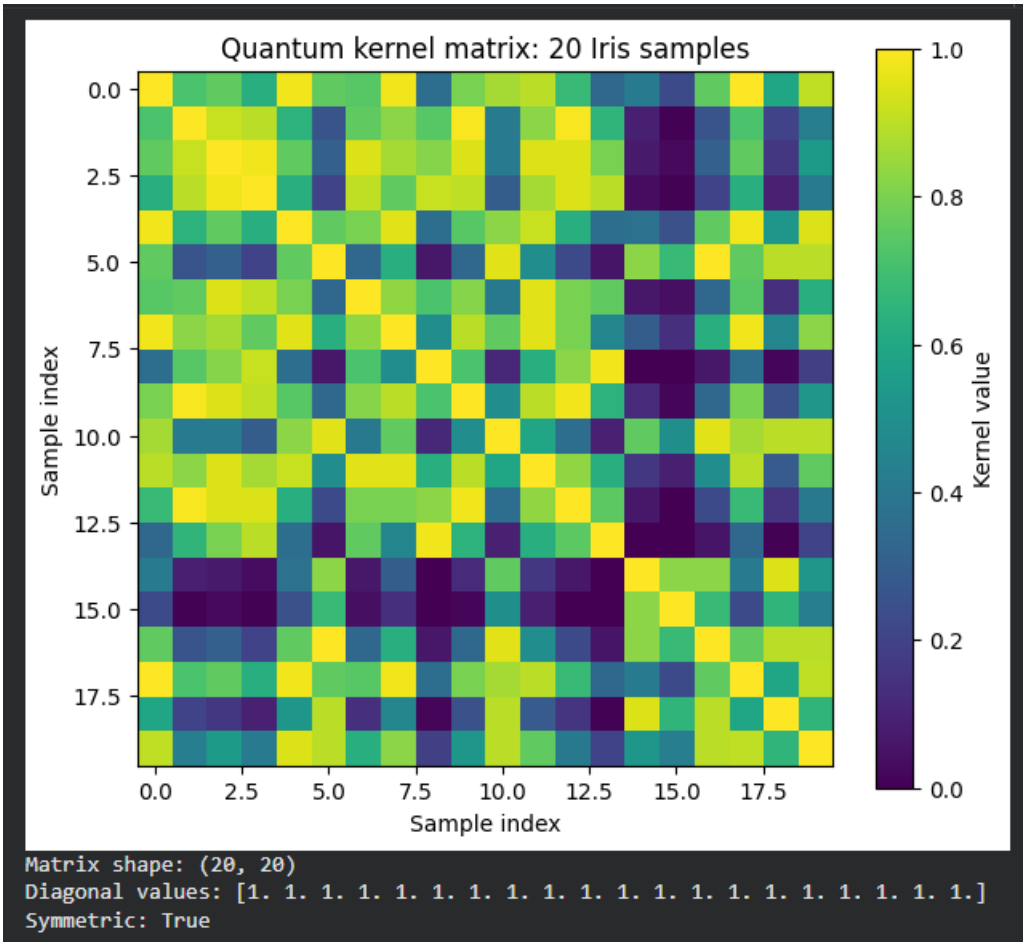
OUTPUT:

```
iris = load_iris()
mask = iris.target < 2
X_raw = iris.data[mask, :2][:20]
y = iris.target[mask][:20]
X = MinMaxScaler().fit_transform(X_raw)

def kernel_matrix(X1, X2=None):
    if X2 is None:
        X2 = X1
    K = np.empty((len(X1), len(X2)))
    states1 = [angle_state(row).data for row in X1]
    states2 = states1 if X2 is X1 else [angle_state(row).data for row in X2]
    for i, a in enumerate(states1):
        for j, b in enumerate(states2):
            K[i, j] = abs(np.vdot(a, b)) ** 2
    return K

K = kernel_matrix(X)
plt.figure(figsize=(7, 6))
plt.imshow(K, cmap='viridis', vmin=0, vmax=1)
plt.colorbar(label='Kernel value')
plt.title('Quantum kernel matrix: 20 Iris samples')
plt.xlabel('Sample index')
plt.ylabel('Sample index')
plt.show()

print("Matrix shape:", K.shape)
print("Diagonal values:", np.diag(K))
print("Symmetric:", np.allclose(K, K.T))
```



Program 8: QSVM: train `SVC(kernel="precomputed")` with the quantum kernel matrix on a training split of the dataset and report the accuracy on the test split. Compare with a classical SVM using the RBF kernel on the same split.

OUTPUT:

```
iris = load_iris()
mask = iris.target < 2
X_raw = iris.data[mask, :2]
y = iris.target[mask]
X = MinMaxScaler().fit_transform(X_raw)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

K_train = kernel_matrix(X_train)
K_test = kernel_matrix(X_test, X_train)

qsvm = SVC(kernel='precomputed')
qsvm.fit(K_train, y_train)
q_pred = qsvm.predict(K_test)
q_acc = accuracy_score(y_test, q_pred)

classical_svm = SVC(kernel='rbf')
classical_svm.fit(X_train, y_train)
c_pred = classical_svm.predict(X_test)
c_acc = accuracy_score(y_test, c_pred)

print(f"Quantum-kernel SVM test accuracy: {q_acc:.3f}")
print(f"Classical RBF SVM test accuracy:  {c_acc:.3f}")
```

```
Quantum-kernel SVM test accuracy: 1.000
Classical RBF SVM test accuracy:  1.000
```

Program 9: Build the swap test circuit for two angle-encoded two-feature data points. Estimate the fidelity from $P(0)$ with at least 8192 shots and compare it with the exact kernel value from Program 6.

OUTPUT:

```python
def swap_test_circuit(x, xp):
    x = np.asarray(x, dtype=float)
    xp = np.asarray(xp, dtype=float)
    n = len(x)
    if len(xp) != n:
        raise ValueError("Feature vectors must have the same length")
    qc = QuantumCircuit(1 + 2*n, 1)
    anc = 0
    first = list(range(1, n+1))
    second = list(range(n+1, 2*n+1))
    for i, value in enumerate(x):
        qc.ry(np.pi * value, first[i])
    for i, value in enumerate(xp):
        qc.ry(np.pi * value, second[i])
    qc.h(anc)
    for a, b in zip(first, second):
        qc.cswap(anc, a, b)
    qc.h(anc)
    qc.measure(anc, 0)
    return qc

x = [0.2, 0.7]
xp = [0.6, 0.1]
qc = swap_test_circuit(x, xp)
print(qc.draw('text'))
result = simulator.run(transpile(qc, simulator), shots=8192).result()
counts = result.get_counts()
p0 = counts.get('0', 0) / 8192
estimated_fidelity = 2 * p0 - 1
exact_kernel = quantum_kernel(x, xp)
print("Counts:", counts)
print("P(0):", p0)
print("Swap-test estimated fidelity:", estimated_fidelity)
print("Exact kernel value:", exact_kernel)
```

```
q_0: ──H──■──■──H──M──
          │  │
q_1: Ry(π/5)─X──┼─────
          │     │
q_2: Ry(7π/10)──X─────
          │
q_3: Ry(3π/5)─X─────
          │
q_4: Ry(π/10)─X─────
          ═══════════
c: 1/═══════════════
                    0
Counts: {'0': 5097, '1': 3095}
P(0): 0.6221923828125
Swap-test estimated fidelity: 0.244384765625
Exact kernel value: 0.22612712429686846
```

Program 10: Quantum clustering: generate a two-cluster dataset of 20 points (for example with `make_blobs`), compute its quantum kernel matrix and cluster the points into two groups using the kernel matrix. Compare the labels with classical k-means and plot both results.

OUTPUT:

```
X_blobs, true_labels = make_blobs(
    n_samples=20, centers=2, n_features=2, cluster_std=1.2, random_state=7
)
X_scaled = MinMaxScaler().fit_transform(X_blobs)
K_blobs = kernel_matrix(X_scaled)

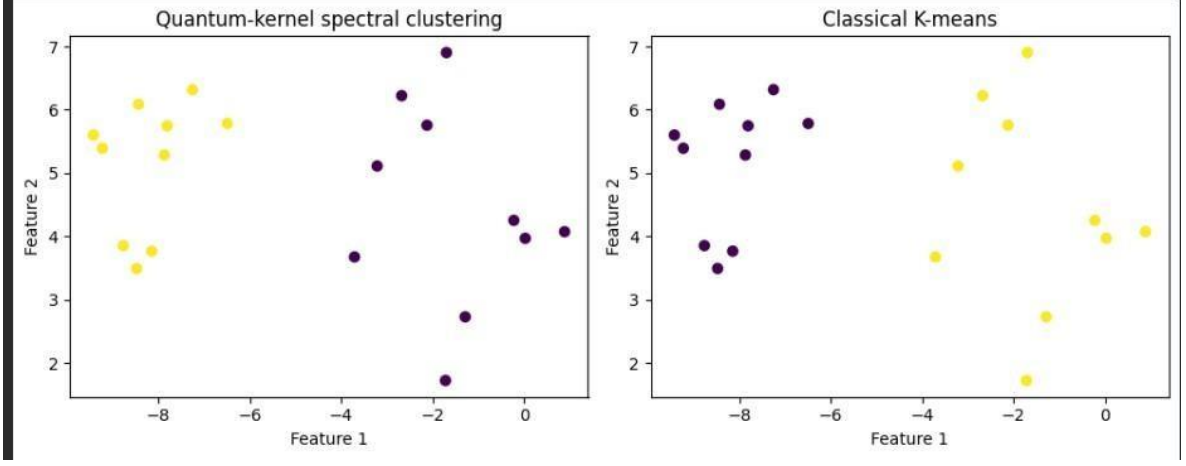
quantum_labels = SpectralClustering(
    n_clusters=2, affinity='precomputed', random_state=7
).fit_predict(K_blobs)
kmeans_labels = KMeans(n_clusters=2, random_state=7, n_init=10).fit_predict(X_scaled)

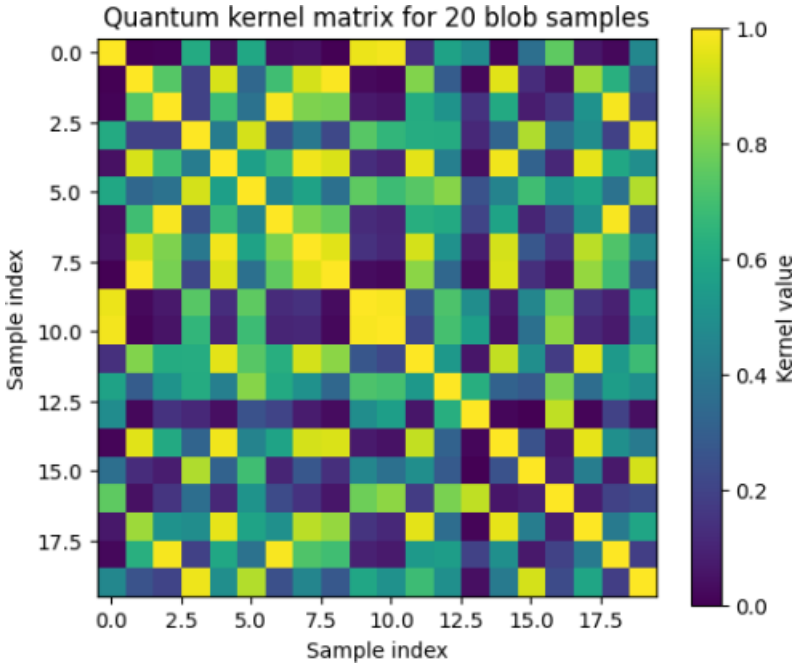
print("Adjusted Rand index: quantum-kernel clustering vs true labels =",
      round(adjusted_rand_score(true_labels, quantum_labels), 3))
print("Adjusted Rand index: K-means vs true labels =",
      round(adjusted_rand_score(true_labels, kmeans_labels), 3))
print("Agreement between clusterings (ARI):",
      round(adjusted_rand_score(quantum_labels, kmeans_labels), 3))

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(X_blobs[:, 0], X_blobs[:, 1], c=quantum_labels)
axes[0].set_title('Quantum-kernel spectral clustering')
axes[0].set_xlabel('Feature 1')
axes[0].set_ylabel('Feature 2')
axes[1].scatter(X_blobs[:, 0], X_blobs[:, 1], c=kmeans_labels)
axes[1].set_title('Classical K-means')
axes[1].set_xlabel('Feature 1')
axes[1].set_ylabel('Feature 2')
plt.tight_layout()
plt.show()

plt.figure(figsize=(6, 5))
plt.imshow(K_blobs, cmap='viridis', vmin=0, vmax=1)
plt.colorbar(label='Kernel value')
plt.title('Quantum kernel matrix for 20 blob samples')
plt.xlabel('Sample index')
plt.ylabel('Sample index')
plt.show()
```

```
Adjusted Rand index: quantum-kernel clustering vs true labels = 1.0
Adjusted Rand index: K-means vs true labels = 1.0
Agreement between clusterings (ARI): 1.0
```





VIVA Q&A:

1. What is data encoding (a feature map), and why is it needed?

Data encoding is the process of converting classical data into a quantum state so that a quantum computer can process it.

2. How many qubits do basis, angle and amplitude encoding need for n features?

- **Basis encoding:** Usually n qubits for n binary features.
- **Angle encoding:** Usually n qubits for n features, using one qubit per feature.
- **Amplitude encoding:** Requires the smallest number of qubits, approximately $\log_2(n)$ qubits for n values. Padding may be needed.

3. In angle encoding, why is the angle taken as πx and not 2πx?

Using πx maps normalized data from 0 to 1 to angles from 0 to π. This gives measurement probabilities ranging from 0 to 1 without repeating within that range. Using 2πx makes the measurement probabilities repeat.

4. In angle encoding, why is the probability of measuring 1 not equal to the data value?

For an Ry gate with angle $\theta = \pi x$, the probability of measuring 1 is:

$P(1) = \sin^2(\pi x/2)$

Therefore, the probability is a nonlinear function of the input value, not the input value itself.

5. Why is amplitude encoding costly even though it uses few qubits?

Amplitude encoding requires preparing a quantum state with specific amplitudes. This can require many quantum gates and complex state preparation, even though it uses relatively few qubits.

6. What is a quantum kernel?

A quantum kernel measures the similarity between two data points by comparing their encoded quantum states. A higher kernel value indicates greater similarity.

7. Why is every diagonal entry of the kernel matrix equal to 1?

Every data point has maximum similarity with itself. Therefore, the diagonal entries of the kernel matrix are 1.

8. In QSVM, which part runs on the quantum computer and which part is classical?

The quantum circuit encodes data and calculates kernel values. The classical computer uses these values to train the SVM and classify data.

9. What does the swap test measure?

The swap test estimates the similarity, or fidelity, between two quantum states using an additional qubit called an ancilla.

10. Does a quantum kernel guarantee better accuracy than a classical kernel?

No. Accuracy depends on the dataset, the quantum feature map, and the model. A classical kernel may perform equally well or better.

RESULT:

Thus, the concepts of basis, angle and amplitude encoding, quantum kernels, QSVM, and the swap test were studied. Their qubit requirements, computational costs, and roles in quantum machine learning were understood.

EVALUATION (for faculty use):

Evaluation method	Prep	Program	Output	Inference	Viva	Timely	Total
Maximum marks	2	8	4	2	3	1	20
Marks awarded							

Faculty signature with date: